

Text Processing & Regular Expressions

Lecture 12

Dr. Colin Rundel

Base R string functions

As you have likely noticed, the individual characters in a string (element of a character vector) cannot be directly accessed. Base R provides a number helpful functions for pattern matching and manipulation of these strings:

- `paste()`, `paste0()` - concatenate strings
- `substr()`, `substring()` - extract or replace substrings
- `sprintf()`, `formatC()` - C-like string formatting
- `nchar()` - counts characters
- `strsplit()` - split a string into substrings
- `grep()`, `grepl()` - regular expression pattern matching
- `sub()`, `gsub()` - regular expression pattern replacement
- + many more - the *See Also* section of the the above functions' documentation to find additional functions.



Strings are not glamorous, high-profile components of R, but they do play a big role in many data cleaning and preparation tasks. The `stringr` package provides a cohesive set of functions designed to make working with strings as easy as possible. ...

`stringr` is built on top of `stringi`, which uses the ICU C library to provide fast, correct implementations of common string manipulations. `stringr` focusses on the most important and commonly used string manipulation functions whereas `stringi` provides a comprehensive set covering almost anything you can imagine.

Fixed width strings - `str_pad()`

```
1 str_pad(10^(0:5), width = 8, side = "left") |>  
2   cat(sep="\n")
```

```
1  
10  
100  
1000  
10000  
1e+05
```

```
1 str_pad(10^(0:5), width = 8, side = "right") |>  
2   cat(sep="\n")
```

```
1  
10  
100  
1000  
10000  
1e+05
```

formatC() (base)

```
1 cat(10^(0:5), sep="\n")
```

```
1
10
100
1000
10000
1e+05
```

```
1 formatC(
2   10^(0:5), digits = 6, width = 6
3 ) |>
4   cat(sep="\n")
```

```
1
10
100
1000
10000
100000
```

```
1 cat(1/10^(0:5), sep="\n")
```

```
1
0.1
0.01
0.001
1e-04
1e-05
```

```
1 formatC(
2   1/10^(0:5), digits = 6, width = 6,
3   format = "fg"
4 ) |>
5   cat(sep="\n")
```

```
1
0.1
0.01
0.001
0.0001
0.00001
```

Whitespace trimming - `str_trim()`, `str_squish()`

```
1 (x = c("  abc" , "ABC  " , " Hello.  World "))
```

```
[1] "  abc"          "ABC  "          " Hello.  World "
```

```
1 str_trim(x)
```

```
[1] "abc"          "ABC"          "Hello.  World"
```

```
1 str_trim(x, side="left")
```

```
[1] "abc"          "ABC  "          "Hello.  World "
```

```
1 str_trim(x, side="right")
```

```
[1] "  abc"          "ABC"          " Hello.  World"
```

```
1 str_squish(x)
```

```
[1] "abc"          "ABC"          "Hello. World"
```

String shortening - `str_trunc()`

```
1 x = "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempo
```

```
1 str_trunc(x, width=60)
```

```
[1] "Lorem ipsum dolor sit amet, consectetur adipiscing elit, ..."
```

```
1 str_trunc(x, width=60, side = "left")
```

```
[1] "...in culpa qui officia deserunt mollit anim id est laborum."
```

```
1 str_trunc(x, width=60, side = "center")
```

```
[1] "Lorem ipsum dolor sit amet, c... mollit anim id est laborum."
```


String wrapping - `str_wrap()`

```
1 cat(x)
```

```
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt
```

```
1 str_wrap(x)
```

```
[1] "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor\ni
```

```
1 str_wrap(x) |> cat()
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
1 str_wrap(x, width=60) |> cat()
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Strings templates - `str_glue()`

This is a simplified wrapper around `glue::glue()` (use the original for additional control).

```
1 paste("The value of pi is" , pi)
```

```
[1] "The value of pi is 3.14159265358979"
```

```
1 str_glue("The value of pi is {pi}")
```

```
The value of pi is 3.14159265358979
```

```
1 paste("The value of tau is" , 2*pi)
```

```
[1] "The value of tau is 6.28318530717959"
```

```
1 str_glue("The value of tau is {2*pi}")
```

```
The value of tau is 6.28318530717959
```

```
1 str_glue_data(  
2   iris |> count(Species),  
3   "{Species} has {n} observations"  
4 )
```

```
setosa has 50 observations  
versicolor has 50 observations  
virginica has 50 observations
```

String capitalization

```
1 x = "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempo
```

```
1 str_to_lower(x)
```

```
[1] "lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in
```

```
1 str_to_upper(x)
```

```
[1] "LOREM IPSUM DOLOR SIT AMET, CONSECTETUR ADIPISCING ELIT, SED DO EIUSMOD TEMPOR IN
```

```
1 str_to_title(x)
```

```
[1] "Lorem Ipsum Dolor Sit Amet, Consectetur Adipiscing Elit, Sed Do Eiusmod Tempor In
```

```
1 str_to_sentence(x)
```

```
[1] "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in
```

Regular Expressions

Regular expressions

A regular expression (shortened as regex or regexp), sometimes referred to as rational expression, is a sequence of characters that specifies a match pattern in text. Usually such patterns are used by string-searching algorithms for “find” or “find and replace” operations on strings, or for input validation. Regular expression techniques are developed in theoretical computer science and formal language theory.

The concept of regular expressions began in the 1950s, when the American mathematician Stephen Cole Kleene formalized the concept of a regular language. They came into common use with Unix text-processing utilities. Different syntaxes for writing regular expressions have existed since the 1980s, one being the POSIX standard and another, widely used, being the Perl syntax.

Source: [Wikipedia](#)

stringr regular expression functions

Function	Description
<code>str_detect</code>	Detect the presence or absence of a pattern in a string.
<code>str_subset</code>	Subset a vector of strings based on the presence of a pattern.
<code>str_locate</code>	Locate the first position of a pattern and return a matrix with start and end.
<code>str_extract</code>	Extracts text corresponding to the first match.
<code>str_match</code>	Extracts capture groups formed by <code>()</code> from the first match.
<code>str_split</code>	Splits string into pieces and returns a list of character vectors.
<code>str_replace</code>	Replaces the first matched pattern and returns a character vector.
<code>str_remove</code>	Removes the first matched pattern and returns a character vector.
<code>str_view</code>	Show the matches made by a pattern.

Many of these functions have variants with an `_all` suffix (e.g. `str_replace_all`) which will match more than one occurrence of the pattern in a string.

Simple Pattern Detection

```
1 text = c("The quick brown" , "fox jumps over" , "the lazy dog")
```

```
1 str_detect(text, "quick")
```

```
[1] TRUE FALSE FALSE
```

```
1 str_detect(text, "o")
```

```
[1] TRUE TRUE TRUE
```

```
1 str_detect(text, "row")
```

```
[1] TRUE FALSE FALSE
```

```
1 str_detect(text, "the")
```

```
[1] FALSE FALSE TRUE
```

```
1 str_detect(  
2   text, regex("the", ignore_case=TRUE)  
3 )
```

```
[1] TRUE FALSE TRUE
```

```
1 str_subset(text, "quick")
```

```
[1] "The quick brown"
```

```
1 str_subset(text, "o")
```

```
[1] "The quick brown" "fox jumps over" "t
```

```
1 str_subset(text, "row")
```

```
[1] "The quick brown"
```

```
1 str_subset(text, "the")
```

```
[1] "the lazy dog"
```

```
1 str_subset(  
2   text, regex("the", ignore_case=TRUE)  
3 )
```

```
[1] "The quick brown" "the lazy dog"
```


Aside - Escape Characters

An escape character is a character which results in an alternative interpretation of the subsequent character(s).

These vary from language to language but for most string implementations `\` is the escape character which is modified by a single following character.

Some common examples:

Literal	Character
<code>\'</code>	single quote
<code>\"</code>	double quote
<code>\\</code>	backslash
<code>\n</code>	new line
<code>\r</code>	carriage return
<code>\t</code>	tab
<code>\b</code>	backspace
<code>\f</code>	form feed

Examples

```
1 print("a\"b")
```

```
[1] "a\"b"
```

```
1 cat("a\"b")
```

```
a"b
```

```
1 print("a\tb")
```

```
[1] "a\tb"
```

```
1 cat("a\tb")
```

```
a    b
```

```
1 print("a\nb")
```

```
[1] "a\nb"
```

```
1 cat("a\nb")
```

```
a  
b
```

```
1 print("a\\b")
```

```
[1] "a\\b"
```

```
1 cat("a\\b")
```

```
a\b
```

Raw character constants

As of v4.0, R has the ability to define raw character sequences which avoids the need for most escape characters, they can be constructed using the `r"(...)"` syntax, where `...` is the raw string.

```
1 print(  
2   "\\int_0^\\infty 1/e^x"  
3 )
```

```
[1] "\\int_0^\\infty 1/e^x"
```

```
1 print(  
2   r"(\int_0^\infty 1/e^x)"  
3 )
```

```
[1] "\\int_0^\\infty 1/e^x"
```

```
1 cat(  
2   "\\int_0^\\infty 1/e^x"  
3 )
```

```
\int_0^\infty 1/e^x
```

```
1 cat(  
2   r"(\int_0^\infty 1/e^x)"  
3 )
```

```
\int_0^\infty 1/e^x
```

RegEx Metacharacters

The power of regular expressions comes from their ability to use special metacharacters to modify how pattern matching is performed.

```
1 . ^ $ * + ? { } [ ] \ | ( )
```

Because of their special properties they cannot be matched directly, if you need to match one you (may) need to escape it first (precede it by `\`).

The problem is that regex escapes live on top of character escapes, so we need to use *two* levels of escapes.

To match	Regex	Literal	Raw
.	\.	"\\."	r"(\.)"
?	\?	"\\?"	r"(\?)"
!	\\!	"\\!"	r"(!)"

Example

```
1 str_detect("abc[def" ,"\[")
```

Error: '\[' is an unrecognized escape in character string supplied as function argument

```
1 str_detect("abc[def" ,"\\[")
```

```
[1] TRUE
```

How do we detect if a string contains a `\` character?

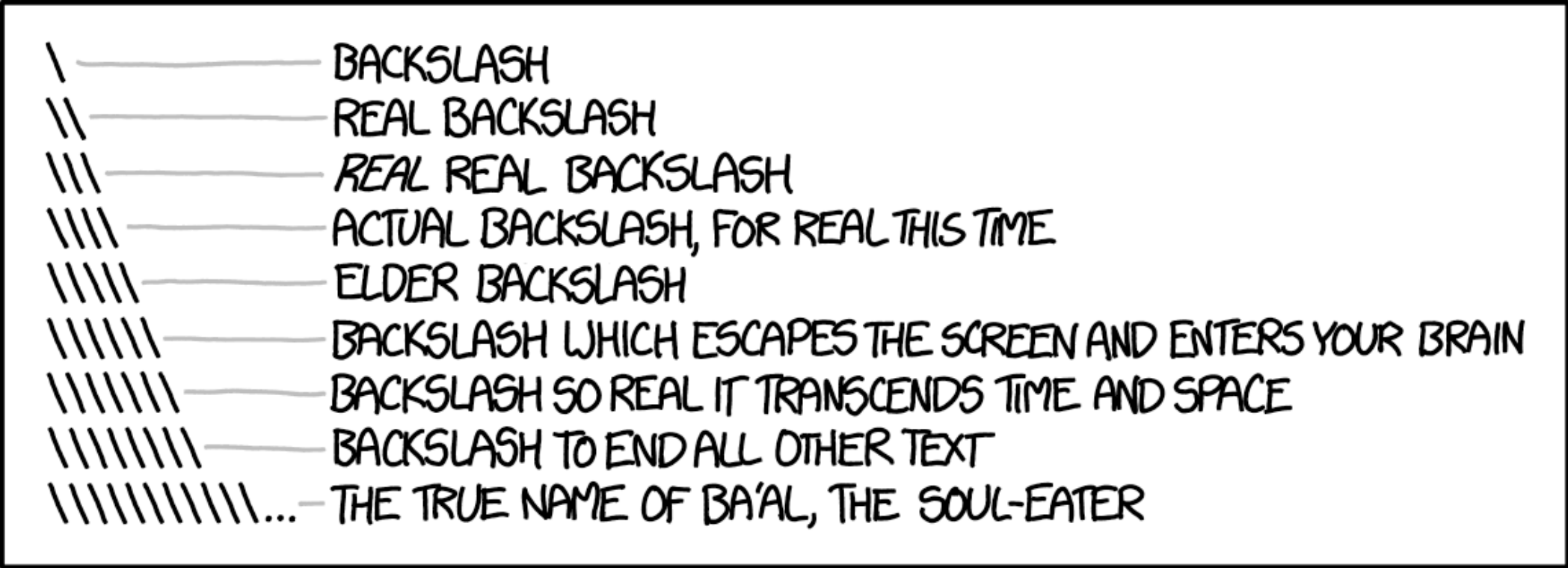
```
1 cat("abc\\def\n")
```

```
abc\def
```

```
1 str_detect("abc\\def" ,"\\\")
```

```
[1] TRUE
```

XKCD's take



\ ————— BACKSLASH
\\ ————— REAL BACKSLASH
\\\ ————— REAL REAL BACKSLASH
\\\\ ————— ACTUAL BACKSLASH, FOR REAL THIS TIME
\\\\\\ ————— ELDER BACKSLASH
\\\\\\\\ ————— BACKSLASH WHICH ESCAPES THE SCREEN AND ENTERS YOUR BRAIN
\\\\\\\\\\ ————— BACKSLASH SO REAL IT TRANSCENDS TIME AND SPACE
\\\\\\\\\\\\ ————— BACKSLASH TO END ALL OTHER TEXT
\\\\\\\\\\\\\\\\... — THE TRUE NAME OF BA'AL, THE SOUL-EATER

Anchors

Sometimes we want to specify that our pattern occurs at a particular location in a string, we indicate this using anchor metacharacters or specific regex escaped characters.

Regex	Anchor
<code>^</code> or <code>\A</code>	Start of string
<code>\$</code> or <code>\Z</code>	End of string
<code>\b</code>	Word boundary
<code>\B</code>	Not word boundary

Anchor Examples

```
1 text = "the quick brown fox jumps over the lazy dog"
```

```
1 str_replace(text, "^the" , "---")
```

```
[1] "--- quick brown fox jumps over the lazy dog"
```

```
1 str_replace(text, "^dog" , "---")
```

```
[1] "the quick brown fox jumps over the lazy dog"
```

```
1 str_replace(text, "the$" , "---")
```

```
[1] "the quick brown fox jumps over the lazy dog"
```

```
1 str_replace(text, "dog$" , "---")
```

```
[1] "the quick brown fox jumps over the lazy ---"
```

```
1 str_replace_all(text, "the" , "---")
```

```
[1] "--- quick brown fox jumps over --- lazy dog"
```


Anchor Examples - word boundaries

```
1 text = "the quick brown fox jumps over the lazy dog"
```

```
1 str_replace_all(text, "\\Brow\\B" , "---")
```

```
[1] "the quick b---n fox jumps over the lazy dog"
```

```
1 str_replace_all(text, "\\brow\\b" , "---")
```

```
[1] "the quick brown fox jumps over the lazy dog"
```

```
1 str_replace_all(text, "\\bthe" , "---")
```

```
[1] "--- quick brown fox jumps over --- lazy dog"
```

```
1 str_replace_all(text, "the\\b" , "---")
```

```
[1] "--- quick brown fox jumps over --- lazy dog"
```

More complex patterns

If there are more than one pattern we would like to match we can use the or (|) metacharacter.

```
1 str_replace_all(text, "the|dog" , "---")
```

```
[1] "--- quick brown fox jumps over --- lazy ---"
```

```
1 str_replace_all(text, "a|e|i|o|u" , "*")
```

```
[1] "th* q**ck br*wn f*x j*mmps *v*r th* l*zy d*g"
```

```
1 str_replace_all(text, "\\ba|e|i|o|u" , "*")
```

```
[1] "th* q**ck br*wn f*x j*mmps *v*r th* lazy d*g"
```

```
1 str_replace_all(text, "\\b(a|e|i|o|u)" , "*")
```

```
[1] "the quick brown fox jumps *ver the lazy dog"
```

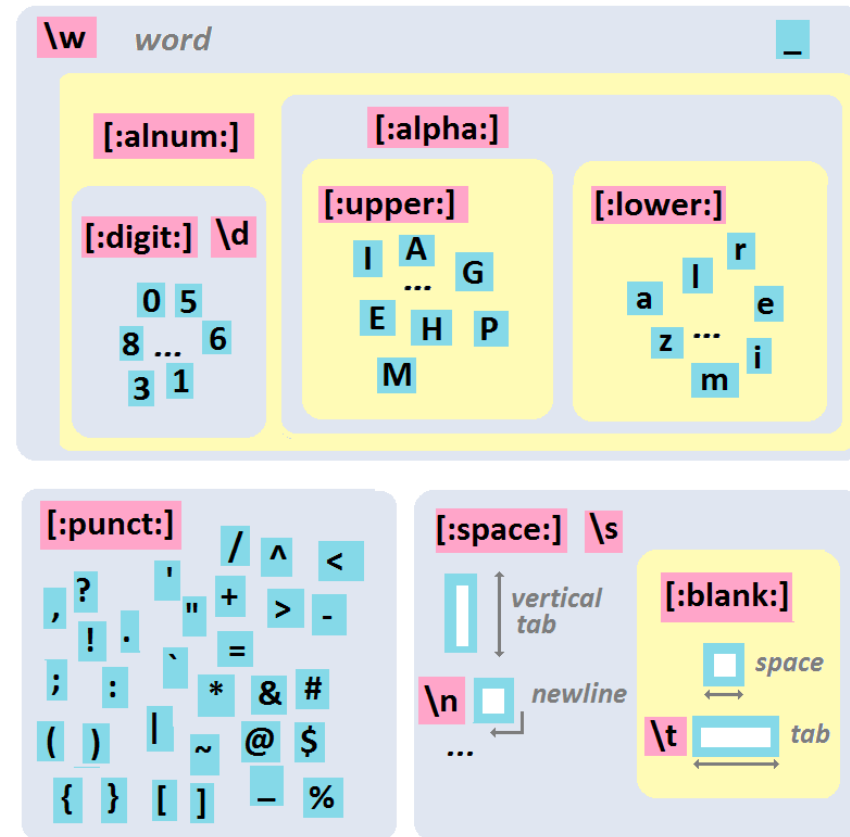
Character Classes

When we want to match whole classes of characters at a time there are a number of convenience patterns built in,

Meta char	Class	Description
.		Any character except new line (<code>\n</code>)
<code>\s</code>	<code>[:space:]</code>	White space
<code>\S</code>		Not white space
<code>\d</code>	<code>[:digit:]</code>	Digit (0-9)
<code>\D</code>		Not digit
<code>\w</code>		Word (A-Z, a-z, 0-9, or <code>_</code>)
<code>\W</code>		Not word
	<code>[:punct:]</code>	Punctuation

A hierarchical view

Predefined character classes



Example

How would we write a regular expression to match a telephone number with the form `(###) ###-####`?

```
1 text = c("apple" , "(219) 733-8965" , "(329) 293-8753")
```

```
1 str_detect(text, "(\d\d\d) \d\d\d-\d\d\d\d")
```

Error: '\d' is an unrecognized escape in character string starting at "(219) 733-8965"

```
1 str_detect(text, "(\\d\\d\\d) \\d\\d\\d-\\d\\d\\d\\d")
```

```
[1] FALSE FALSE FALSE
```

```
1 str_detect(text, "\\(\\d\\d\\d\\) \\d\\d\\d-\\d\\d\\d\\d")
```

```
[1] FALSE TRUE TRUE
```

Classes and Ranges

We can also specify our own classes using square brackets, to simplify these classes ranges can be used for contiguous characters or numbers.

Class	Type
[abc]	Class (a or b or c)
[^abc]	Negated class (not a or b or c)
[a-c]	Range lower case letter from a to c
[A-C]	Range upper case letter from A to C
[0-7]	Digit between 0 to 7

Example

```
1 text = c("apple" , "(219) 733-8965" , "(329) 293-8753")
```

```
1 str_replace_all(text, "[aeiou]" , "*")
```

```
[1] "*pp*l*"          "(219) 733-8965" "(329) 293-8753"
```

```
1 str_replace_all(text, "[13579]" , "*")
```

```
[1] "apple"           "(2**) ***-8*6*" "(*2*) 2**-8***"
```

```
1 str_replace_all(text, "[1-5a-f]" , "*")
```

```
[1] "*pp*l*"          "(**9) 7**-896*" "(**9) *9*-87**"
```

```
1 str_replace_all(text, "[^1-5a-f]" , "*")
```

```
[1] "a***e"           "*21*****33*****5" "*32***2*3***53"
```

Exercises 1

For the following vector of randomly generated names, write a regular expression that,

- detects if the person's first name starts with a vowel (a,e,i,o,u)
- detects if the person's last name starts with a vowel
- detects if either the person's first or last name start with a vowel
- detects if neither the person's first nor last name start with a vowel

```
c("Jeremy Cruz", "Nathaniel Le", "Jasmine Chu", "Bradley Calderon Raygoza",  
  "Quinten Weller", "Katelien Kanamu-Hauanio", "Zuhriyaa al-Amen",  
  "Travale York", "Alexis Ahmed", "David Alcocer", "Jairo Martinez",  
  "Dwone Gallegos", "Amanda Sherwood", "Hadiyya el-Eid", "Shaimaaa al-Can",  
  "Sarah Love", "Shelby Villano", "Sundus al-Hashmi", "Dyani Loving",  
  "Shanelle Douglas")
```


Quantifiers

Attached to literals or character classes these allow a match to repeat some number of time.

Quantifier	Description
*	Match 0 or more
+	Match 1 or more
?	Match 0 or 1
{3}	Match Exactly 3
{3,}	Match 3 or more
{3,5}	Match 3, 4 or 5

Example

How would we improve our previous regular expression for matching a telephone number with the form `(###) ###-####`?

```
1 text = c("apple" , "(219) 733-8965" , "(329) 293-8753")
```

```
1 str_detect(text, "\\(\\d\\d\\d\\) \\d\\d\\d-\\d\\d\\d\\d")
```

```
[1] FALSE TRUE TRUE
```

```
1 str_detect(text, "\\(\\d{3}\\) \\d{3}-\\d{4}")
```

```
[1] FALSE TRUE TRUE
```

```
1 str_extract(text, "\\(\\d{3}\\) \\d{3}-\\d{4}")
```

```
[1] NA "(219) 733-8965" "(329) 293-8753"
```

Greedy vs non-greedy matching

What went wrong here?

```
1 text = "<div class='main'> <div> <a href='here.pdf'>Here!</a>
```

```
1 str_extract(text, "<div>.*</div>")
```

```
[1] "<div> <a href='here.pdf'>Here!</a> </div> </div>"
```

If you add `?` after a quantifier, the matching will be *non-greedy* (find the shortest possible match, not the longest).

```
1 str_extract(text, "<div>.*?</div>")
```

```
[1] "<div> <a href='here.pdf'>Here!</a> </div>"
```

Groups

Groups allow you to connect pieces of a regular expression for modification or capture.

Group	Description
(a b)	match literal “a” or “b” , group either
a(bc)?	match “a” or “abc” , group bc or nothing
(abc)def(hig)	match “abcdefhig” , group abc and hig
(?:abc)	match “abc” , non-capturing group

Example

```
1 text = c("Bob Smith" , "Alice Smith" , "Apple")
```

```
1 str_extract(text, "^[:alpha:]+")
```

```
[1] "Bob"    "Alice"  "Apple"
```

```
1 str_match(text, "^[:alpha:]+")
```

```
      [,1]  
[1,] "Bob"  
[2,] "Alice"  
[3,] "Apple"
```

```
1 str_extract(text, "^(:alpha:)+ [:alp")
```

```
[1] "Bob Smith"    "Alice Smith" NA
```

```
1 str_match(text, "^(:alpha:)+ [:alp")
```

```
      [,1]      [,2]  
[1,] "Bob Smith" "Bob"  
[2,] "Alice Smith" "Alice"  
[3,] NA          NA
```

```
1 str_extract(text, "^(:alpha:)+ (:alp")
```

```
[1] "Bob Smith"    "Alice Smith" NA
```

```
1 str_match(text, "^(:alpha:)+ (:alp")
```

```
      [,1]      [,2]      [,3]  
[1,] "Bob Smith" "Bob"    "Smith"  
[2,] "Alice Smith" "Alice" "Smith"  
[3,] NA          NA      NA
```

How not to use a RegEx

Validating an email address:

```
(?:[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\.[a-z0-9!#$%&'*/+=?^_`{|}~-]+  
(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-\x0  
@(?:((?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.[a-z0-9](?:[a-z0-9-]*[a  
(?:((?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}  
(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?|[a-z0-9-]*[a-z0-9]):  
(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[  
[\x01-\x09\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\[
```

Exercise 2

```
1 text = c(  
2   "apple" ,  
3   "219 733 8965" ,  
4   "329-293-8753" ,  
5   "Work: (579) 499-7527; Home: (543) 355 3679"  
6 )
```

- Write a regular expression that will extract *all* phone numbers contained in the vector above.
- Once that works use groups to extract the area code separately from the rest of the phone number.